Solución primer parcial setiembre 2026

Ejercicio 1

a) //Pos-cond: retorna el carácter que aparece mas veces en la lista, en caso de //que haya 2 o más caracteres con la misma cantidad de ocurrencias retorna //el primero

b) Si la lista tiene (n) nodos, el orden es **O(n²)**.

El ciclo exterior recorre la lista una vez. Para **cada nodo**, el ciclo interior vuelve a recorrer la lista completa para contar cuántas veces aparece su carácter. Por eso, al crecer la lista, se repite el recorrido completo para cada nodo.

c) Sí. Como los elementos son caracteres, podemos contar las apariciones de cada valor posible de `unsigned int` en un arreglo. Luego buscamos la frecuencia máxima y recorremos la lista para devolver el primer carácter que tenga esa frecuencia, conservando así el criterio de desempate de la función original.

El nuevo orden es **O(n)**, suponiendo que la cantidad de valores posibles de `char` es fija. Se recorren la lista dos veces y el arreglo de frecuencias tiene un tamaño constante.

```
char funcion(Lista lista) {
    int frecuencias[UCHAR_MAX + 1] = {};
    nodo_lista *p = lista;
    while (p != NULL) {
        unsigned int indice = p->elem;
        frecuencias[indice]++;
        p = p->sig;
    }
    int maxima = 0;
    for (int i = 0; i <= UCHAR_MAX; i++)
        if (frecuencias[i] > maxima)
            maxima = frecuencias[i];
    p = lista;
    while (p != NULL) {
        unsigned int indice = p->elem;
        if (frecuencias[indice] == maxima)
            return p->elem;
        p = p->sig;
    }
    return '\0'; // Caso no alcanzable si la lista no está vacía.
}
```

Ejercicio 2:

```
void eliminarConsecutivos(Lista &lista) {
    if (lista != NULL) {
        if (lista->sig != NULL &&
            lista->elem == lista->sig->elem) {

            nodo_lista *borrar = lista->sig;
            lista->sig = borrar->sig;
            delete borrar;

            eliminarConsecutivos(lista);
        } else {
            eliminarConsecutivos(lista->sig);
        }
    }
}
```

Ejercicio 3

```
void eliminarEntreIguales(Lista &lista) {
    nodo_lista *actual = lista->inicio;
    while (actual != NULL) {
        if (actual->ant != NULL &&
                actual->sig != NULL &&
                    actual->ant->elem == actual->sig->elem) {
            nodo_lista *anterior = actual->ant;
            nodo_lista *siguiente = actual->sig;
            anterior->sig = siguiente;
            siguiente->ant = anterior;
            delete actual;
            actual = anterior;
        } else {
            actual = actual->sig;
        }
    }
}
```